



TECH2

Week 6 Workshop

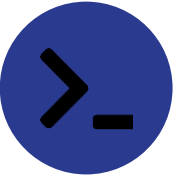
Virtual Environments with conda



What We Did in the Lecture

- In the lecture we installed numpy, matplotlib and sympy — straight into the base environment.
- That was deliberate. It is also the one thing you should never do.
- base is the environment conda sets up for itself. Anything installed there is shared by every project on your computer.
- Today: why that is a problem, and what to do instead.

***Today's goal:** leave base alone, and give every project an environment of its own.*



conda and Miniforge

- Miniforge gives you two things: Python, and conda.
- conda is a package manager — it installs packages, and works out which versions fit together.
- conda is also an environment manager — it creates isolated workspaces for your projects.
- Miniforge is deliberately minimal: base starts nearly empty, and packages come from conda-forge, a large community-maintained repository.

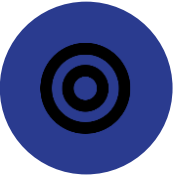
Where packages live: *conda-forge.org* — every conda install and conda create downloads its packages from there.



What Is a Virtual Environment?

- An isolated folder with its own copy of Python and its own set of packages.
- Each project gets one. Installing a package into one environment cannot affect any other.
- Environments live centrally, inside your Miniforge folder — not inside your project folder.
- You switch between them with a single command: `conda activate <name>`.

The prompt tells you where you are: (base) means you are in the base environment, (my_env) means you are in yours.



Why It Matters

- **Avoid conflicts** — project A needs numpy 1.25, project B needs numpy 1.21. In base, installing one breaks the other.
- **Keep base clean** — a broken base installation affects everything you do, in every project.
- **Reproducible setups** — an environment can be written down and rebuilt exactly, by anyone.
- **Multiple Python versions** — different projects can use different versions of Python itself.
- **Safe testing** — experiment freely, and just delete the environment if it goes wrong.

The whole idea: one project's mess stays inside that project.



Your Toolkit

- 1 `conda --version` — checks that conda is installed and working
- 2 `conda env list` — shows every environment on your computer
- 3 `conda list` — shows what is installed in base right now. You should see numpy, matplotlib and sympy from the lecture.
- 4 `conda config --show channels` — shows where conda gets packages from. It should say conda-forge and nothing else.

All commands are in `exercises.md` — copy and paste them from there rather than typing them by hand.



Creating an Environment

- `conda create --name my_env python=3.14 pandas`
- `--name my_env` — what to call it. The project name is usually a good choice.
- `python=3.14` — always specify a Python version, or conda picks one for you.
- `pandas` — any packages you want from the start. List as many as you like, or none at all.
- Creating an environment does not start using it — for that you need `conda activate my_env`.
- You must also make sure to select the correct environment in VS Code.

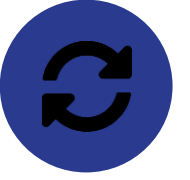
***ModuleNotFoundError** for a package you know you installed? You are almost certainly in the wrong environment.*



Exercise 1 — Create an Environment and Use It

- 1 Part A: create tech2-test with Python 3.14 and pandas, then activate it
- 2 Check that pandas is there — then deactivate and run the same check in base
- 3 Part B: write a short script that imports pandas, and run it under tech2-test
- 4 Switch the interpreter to base and run exactly the same script again

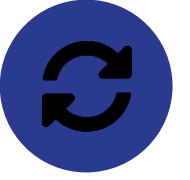
Expect the last step to fail — that is the point of the exercise. Commands are in exercises.md.



Sharing an Environment

- An environment can be written down as a small text file: `environment.yml`.
- **`conda env export --from-history > environment.yml`**
- The file lists the environment's name, its channels, and the packages you asked for — that is all anyone needs.
- Anyone can then rebuild it with `conda env create --file environment.yml`.

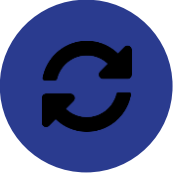
***Use `--from-history`.** A plain export also lists every automatically-installed dependency, pinned to versions that may only exist on your computer.*



Exercise 2 — Export, Delete, Rebuild

- 1 Export tech2-test to environment.yml, and open the file in VS Code
- 2 Deactivate the environment, then delete it entirely
- 3 Confirm with conda env list that it really is gone
- 4 Rebuild it from the file, and check that pandas is back

You just destroyed an environment and recreated it from a text file. That file is the whole point of today.



Exercise 3 — Your Part 2 Environment

- 1 Create the environment straight from a URL — no downloading first
- 2 Check that it appears in `conda env list`
- 3 Activate it, and confirm that `pandas` is installed — but `sympy` is not
- 4 Run the import check in a notebook with `TECH2` as its kernel — then keep this environment for the rest of the course

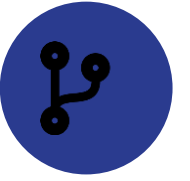
***`conda env create --file`** accepts a web address just as happily as a file on your own computer.*



Exercise 4 — Recreate Someone Else's Project

- 1 Download and unzip temp-conversion-app.zip
- 2 Open its environment.yml and see what the project needs
- 3 Build the environment from that file
- 4 Open the notebook, select temp-conversion-app as its kernel, and click Run All — then open the link below the last cell

You are about to run a project built with a package you have never used, without installing anything by hand. That is what an environment file buys you.



conda vs. pip

- pip is Python's own package installer, and it works inside conda environments too.
- **Prefer conda** — it checks that all your packages fit together before installing anything.
- **Use pip when conda cannot help** — some packages are not on [conda-forge](#), and new releases reach [PyPI](#) first.
- **Install conda packages first, then pip** — installing with conda afterwards can overwrite what pip did.

***Careful:** `--from-history` does not record pip installs. If you used pip, add those packages to `environment.yml` yourself.*



Best Practices

- One environment per project.
- Never install into base.
- Give environments clear, descriptive names — usually the project name.
- Pin the versions that actually matter to the project.
- Export an `environment.yml` and keep it alongside your code.
- Delete environments you no longer use — they take up real disk space.



What You've Built Today

- tech2-test — created, exported, deleted, and rebuilt from a file
- Your Part 2 environment — ready for pandas and matplotlib
- temp-conversion-app — someone else's project, running on your machine
- The habit to keep: one environment per project, and never touch base

**Next week: Part 2 — working with data.
From now on, you will work inside the TECH2 environment.**